

Introduction to Deep Learning

Session 7: Generalization & Regularization

May 2026

Magda Gregorová - magda.gregorova@thws.de



Licensed according to CC-BY-SA 4.0

Lecture Overview

1. Training Stability

2. Summary

Training Stability

Regularisation: can use large, overparameterised networks without overfitting :)

New problem: deep NNs training instabilities - more layers amplify small issues :(

- **Internal covariate shift** - activation distributions shift as parameters update
- **Vanishing/exploding grads** - signals decay or amplify exponentially through layers
- **Dying ReLU** - neurons get stuck outputting zero permanently

Not overfitting problems - regularisation does not help here

Problem: as weights update, the input distribution to each layer changes continuously

- Layer l learns to process inputs with a certain distribution
- Layer $l - 1$ updates its weights - distribution shifts
- Layer l must constantly re-adapt - a moving target
- Effect amplifies with depth - later layers more instability

Consequences:

- Slower convergence
- Need for lower learning rates
- High sensitivity to initialisation

Solution: explicitly normalise activations at each layer - **Batch Normalisation**

Idea: normalise activations within each mini-batch -> distributions stable across layers

$$\hat{h}_j = \frac{h_j - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}} \quad \tilde{h}_j = \gamma \hat{h}_j + \beta$$

μ_B, σ_B^2 - mini-batch mean and variance γ, β - learnable scale and shift

`model.train():`

- Normalise using mini-batch stats
- Accumulate running mean and var

`model.eval():`

- Normalise using running stats
- Deterministic - no batch dependence

Benefits: stable training, allows higher learning rates, less sensitive to initialisation

Side effect: mini-batch noise introduces mild regularisation - but not main motivation

Vanishing and Exploding Gradients

Backprop multiplies Jacobians backwards - effect compounds, earliest layers affected most

$$\frac{\partial \mathcal{L}}{\partial \theta^{(1)}} = \frac{\partial \mathcal{L}}{\partial \mathbf{h}^{(L)}} \cdot \frac{\partial \mathbf{h}^{(L)}}{\partial \mathbf{h}^{(L-1)}} \cdots \frac{\partial \mathbf{h}^{(2)}}{\partial \mathbf{h}^{(1)}} \cdot \frac{\partial \mathbf{h}^{(1)}}{\partial \theta^{(1)}}$$

Vanishing (factors < 1)

- Gradient $\rightarrow 0$ at earliest layers
- Early layers stop learning
- Deep NNs effectively shallow

Exploding (factors > 1)

- Gradient $\rightarrow \infty$ at earliest layers
- Single large step destroys weights
- Training diverges

Solutions: Xavier/He init (Session 5); batch norm; gradient clipping (RNN, Transformers)

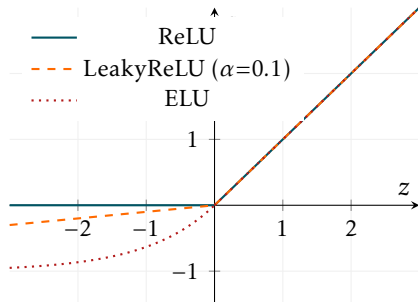
Alarm: ReLU has zero gradient for all negative pre-activations - **half the input space**

$$\frac{\partial \text{ReLU}(z)}{\partial z} = \begin{cases} 1 & z > 0 \\ 0 & z \leq 0 \end{cases}$$

Relief: neuron dies if pre-activation negative for **all** training examples **simultaneously**

- Pre-activations vary across examples - some positive, some negative
- Average gradient is nonzero - diversity in data saves us
- **ReLU remains solid baseline**

Active research area: many alternatives handle the negative region differently



- **LeakyReLU** - small negative slope α
- **ELU** - smooth exponential region
- **GELU, Swish, Mish** - smooth, used in modern architectures

No universal winner - **ReLU** and **LeakyReLU** cover most practical needs

Summary

Randomness pervasive in deep learning training - not a nuisance but a feature

Source	Where	Effect
Weight initialisation	start of training	which minimum is found
Data shuffling	each epoch	different batches, prevents order memorisation
Mini-batch SGD	every update step	implicit regularisation, flat minima
Dropout	forward pass	ensemble effect, prevents co-adaptation
Data augmentation	data loading	invariance, population coverage

Embracing randomness rather than eliminating it is part of what makes deep networks generalise well

Generalisation:

- True goal - low loss on population
- Generalisation gap - monitor via validation loss
- Deep NNs generalise surprisingly well
 - flat minima, double descent

Regularisation:

- Weight decay - explicit penalty
- Dropout - random subnetworks
- Early stopping - cut training time
- Data augmentation - most powerful

Training stability:

- Batch norm - stable activation distribs
- Gradient clipping - stop exploding grads
- LeakyReLU / ELU - avoid dying ReLU

In PyTorch:

- `nn.Dropout(p)`
- `nn.BatchNorm1d`
- `weight_decay` in `torch.optim`
- `model.train()` / `model.eval()`